大型软件工程实践 - 教学大纲

(2026 July 15-19)

Term: Summer Intensive (16 Hours Total)
Prerequisites: Open Minds, High Tolerance for Ambiguity

Course Philosophy: The Shift in the AI Era

In the AI era, writing good code is no longer the only scarce skill. The challenge software engineers face is knowing what problem should be solved, what assumptions are hidden, what failure modes emerge when systems interact, and when an AI-generated answer is plausible but wrong.

This course trains students to think like engineers responsible for large-scale systems: mapping feedback loops, challenging assumptions, reasoning from first principles, negotiating interfaces, and defending designs under uncertainty.

I. 课程目标 Core Learning Objectives

This course aims at approaching software engineering implementation through Inquiry-Based Learning.
Rather than acquiring technical skills or knowledge of frameworks, this course focuses entirely on cultivating three vital intellectual & cognitive habits.

Systems Thinking: Moving past simple cause-and-effect to map complex feedback loops and emergent properties.

First Principles & Deep Discernment: Developing the sharp perception to look past surface-level correctness, question foundational assumptions, and separate signal from noise.

Cognitive Agility & Emotional Intelligence: Learning to collaborate, negotiate (contracts)boundaries, and pivot fluidly when a volatile reality destroys your initial plan.

II. Syllabus

Duration: 5 Days (16 Hours Total)
Format: Inquiry-Based Learning (IBL), No-Coding, Whiteboarding, AI as Adversary Role

---

Day 1. The Pivot — Reductionism to Holism, Emergence, Hidden Assumptions

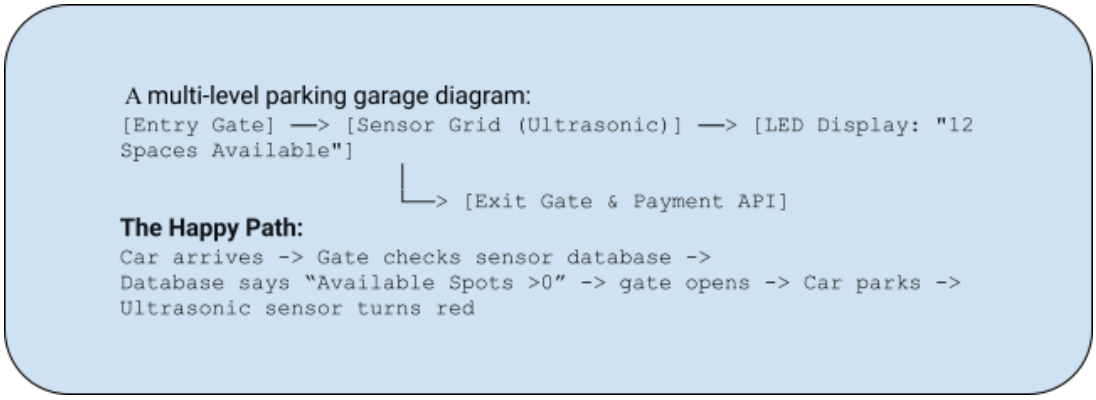
Time: 3 Hours
Focus: Transitioning from "finding correct solutions" to "questioning structural logic."
Student output: “Failure assumption map” with at least 5 hidden assumptions

Session Breakdown:

Hour 1: Contrast of two paradigms

- "Machine view" (deterministic) with the "Ecosystem view" (emergent).
- Reductionist Logic vs Systems Thinking in Software Engineering
- Shift focus from simple Judgment to deep Discernment (uncovering hidden assumptions).

In-Class Guided Run (25 minutes) : A simple parking garage design:



Concept discussion:

Reductionism	Holism / Systems Thinking
1. Handling Bugs and Failures (The Debugging Paradox)	
“What is this thing made of?” Assumes linear causality.	*"What is this thing a part of?"* — and — *"What happens when the parts interact?"* - Assumes circular causality.

Bug A causes Failure B. Find the exact line of code, the faulty variable, or the crashed microservice, and fix it in isolation.	Failure B emerges from the *interactions* between services (e.g., timeout + retry storm + garbage collection pause). Fixing one service won't work; you must redesign the interaction pattern (e.g., circuit breakers, backpressure).
Example: "The database query is slow. Let's add an index.	Example: "The database query is slow because the orchestration layer calls it 500 times in a loop. Let's change the orchestration logic and introduce caching *and* rate-limiting *together*."
2. Analyzing Distributed Systems (The Classic "Fallacies of Distributed Computing")	
Reductionism says: *"Network latency is 10ms. Database latency is 5ms. So my total response time is 15ms."*	Systems thinking says: *"Under 80% load, latency is 15ms. But at 81% load, the network switch starts queueing, retries spike, and latency jumps to 2,000ms. This non-linear behavior cannot be predicted by summing up the parts. I must build bulkheads, backpressure, and observability to sense the whole state in real-time."*

Hours 2: Case Study

Case Study 1: Netflix (Systems Thinking / Holism in Practice)

The Context: Netflix streams to 260+ million members across 190+ countries, running on AWS across millions of microservices instances.

The Systems Thinking Approach (Chaos Engineering):

Netflix explicitly rejects the reductionist idea that *"if each service is 99.99% available, the whole system is 99.99% available."* They know this is mathematically false because dependencies multiply failure rates.

Instead, they built

- Built Chaos Monkey (and the full Simian Army).
- Identified the Holistic Feedback Loop
- Applied Systems Thinking Fixes

The Result: Netflix can lose an entire AWS Availability Zone (AZ)—hundreds of services dying simultaneously—and the holistic experience (pressing play and watching a video) remains uninterrupted for 99.99% of users. They don't care which part died; they care that the *whole streaming session* survives.

Hours 3: Challenge-0 — The Deconstructive Flash-Mob

- **The Setup:** Teams analyze an optimized diagram of a simple deterministic system.
- **The AI Adversary:** Teams input their list of architectural assumptions into DeepSeek/ChatGPT, prompting: *"Act as a cynical Senior Architect. What blind spots did we miss? Force us to see how this simple system fails in the messy real world."*
- **The Output:** Whiteboard a map of the newly exposed system weaknesses.
- 📝 **End-of-Day Reflection Prompt:** *"What is the difference between a system passing a test on paper versus surviving a messy reality? What did the AI perceive about our human/environmental assumptions that we were totally blind to?"*

Day 2. The Ecosystem — Mapping Feedback Loops, Local Optimization, Non-linear Behavior

Time: 3 Hours
Focus: Cultivating **Systems Thinking** through non-linear consequences.
Student output: Causal loop diagram with at least 2 reinforcing loops and 2 balancing loops

Session Breakdown

- **Hour 1: The Physics of Systems (Core Concepts)**
    - Introduce balancing loops, reinforcing loops, and emergent properties.
    - Discuss why local optimizations routinely destroy global ecosystems.
    - (add: contrast between Reductionism vs. Systems approaches)
- **Hours 2 & 3: Challenge 1 — The "Invisible" Ripple**
    - **The Setup:** Deliver the Ride-Sharing notification feature blueprint. Teams map the initial architecture.
    - **The AI Adversary:** Teams feed their design to the AI: *"Act as a crowd of millions of emotionally volatile human users. How do we accidentally trigger a passenger revolt, driver shortages, or surge-pricing chaos with this feature?"*
    - **The Output:** Teams must map the cascading human failures on whiteboards and architect behavioral mitigation loops.
    - 📝 **End-of-Day Reflection Prompt:** *"When our system interacted with human psychology, what surface-level metrics did we focus on, and what deeper architectural truth did the AI adversary force us to uncover?"*

Day 3: First Principles — Deconstructing AI Systems, Essential vs. Accidental Complexity

Time: 3 Hours
Focus: Cultivating **Metacognition & Discernment** when working alongside AI tools.
Student output: "Constraint-first architecture" showing immutable constraints before system design

Session Breakdown

- **Hour 1: The Trap of the Black Box**
    - Discuss why trial-and-error debugging fails when dealing with AI-generated structures or legacy codebases.
    - Define **First-Principles Thinking**: stripping problems down to immutable constraints.
- **Hours 2 & 3: Challenge 2 — The Haunted AI Legacy System**
    - **The Setup:** Teams are presented with the chaotic airline scheduling logic map. They are forbidden from trying to "patch" the rules.

- **The AI Adversary:** Teams prompt the AI to defend its complex codebase: *"We hypothesize that your scheduling logic relies on a false, fragile assumption. Defend your architecture using reductionist arguments, and challenge us to find the structural flaw."*
- **The Output:** Whiteboard a brand-new scheduling framework derived strictly from fundamental physical constraints (pilot hours, physical plane locations).
- 📝 **End-of-Day Reflection Prompt:** *"How did we separate the AI's logical 'hallucinations' from actual structural truths? How does anchoring a design in First Principles protect us from legacy bloat?"*

Day 4: Friction & Agility — Managing Human Systems, Conway's Law, Ownership, Escalation

Time: 3 Hours
Focus: Cultivating **Agility and Emotional Intelligence** under resource constraints.
Student output: One-page API/social contract between teams, including failure modes

Session Breakdown

- **Hour 1: Interfaces, Contracts, and Human Chaos**
    - Explore how system architecture is deeply bound by human negotiation, trust, and communication.
- **Hours 2 & 3: Challenge 3 — Smart City Power Grid Game**
    - **The Setup:** Divide the class into 4 municipal sector teams pulling from a shared, AI-managed power grid during a massive heatwave.
    - **The AI Adversary:** While teams negotiate, they query a shared AI: *"We have designed this collaborative API contract to balance our power needs. Act as a malicious node or an extreme weather event trying to exploit our human agreement. Where is our contract weak?"*
    - **The Output:** Teams must dynamically rewrite their cross-team interface contract under live, AI-generated pressure.
    - 📝 **End-of-Day Reflection Prompt:** *"What was the single biggest unspoken assumption our team made about the other sectors that the AI adversary*

aggressively corrected? How did we manage team friction to pivot our contract?"

Day 5: Reality Synthesis — Final System Defense, Error Budget, Resilience, Tradeoff

Time: 4 Hours
Focus: Demonstrating synthesis of cognitive habits, agility, and discernment.
Student output: Before/after architecture plus 3 pivots made under pressure

Session Breakdown

- **Hours 1 & 2: The Final Architectural Defense**
    - Each group presents one of their revised architectures from the week to a panel of human instructors and guest architects.
    - **The Human + AI Adversary:** The panel throws live "Chaos Variables" at their whiteboards, backed by real-time adversarial prompts from DeepSeek to challenge the team's live defenses.
    - Grading is based on how fast the team pivots (**Agility**) and maps structural truths (**Discernment**) under live pressure.
- **Hour 3: Final Course Reflection**
    - 📝 **Final Course Reflection Prompt:** *"Reflecting on the past 5 days, how has your definition of an 'engineer' shifted? When you return to code-heavy classes, how will you use discernment to test reality rather than just seeking 'correct' solutions?"*

III. Group Exercises: Reductionism vs. Systems/First-Principles

Students will work together during class on the following exercises: **Think → Pair → Team → Public defense**

First, each student writes silently for 3 minutes. Then they compare with one partner. Then the team consolidates.
For example, in the ride-sharing exercise, instead of asking the class to discuss immediately, give this sequence:

1. Individually write 3 possible unintended consequences.
2. Pair with another student and choose the most dangerous one.
3. Team draws one feedback loop.
4. Team asks AI to attack the loop.
5. Team decides whether the AI criticism is valid or hallucinated.
6. Team presents one revised mitigation.

Rules: students can use AI as their adversary, or use two AI models, one as their co-pilot and one as adversary.

The "Discernment" Checkpoint for Students

Warning students about a key trap: **AI models can sometimes be too agreeable or overly critical without substance.**
Teach students that they must practice *discernment* on the AI itself. If DeepSeek says, *"Your design will fail because of X,"* the students must not blindly accept it. They must ask: *"Is X a fundamental truth of the universe (First Principles), or is the AI just hallucinating a surface-level error (Judgment)*

Every AI criticism must be classified into one of three buckets:

Bucket	Meaning	Student action
Fundamental constraint	Based on physics, law, human limits, economics, or unavoidable system behavior	Must address
Plausible risk	Could happen, but depends on assumptions	Must test or monitor
AI noise	Vague, exaggerated, unsupported, or generic	Reject or refine prompt

Group Exercise 1: The "Invisible" Ripple (Ecosystem Feedback Loops)

The Scenario:

Students are handed an architectural map of a massive ride-sharing platform. The business team wants to implement a single, straightforward feature: *"If a user's rider rating drops below 3.5 stars, automatically send them a polite warning notification with tips on how to be a better passenger."*

- **The Reductionist Trait (The Trap):** A reductionist looks at this as an isolated component. Inputs (User Rating < 3.5) \(\rightarrow \) Processing (Trigger Notification) \(\rightarrow \) Output (Send Message). It passes a mental "unit test."
- **The Systems Reality (The Climax):** When deployed, millions of low-rated riders receive the notification simultaneously. Instead of changing their behavior, a statistically significant portion of them get angry and aggressively downvote their next drivers out of spite. This triggers an automated system that suspends those drivers for poor ratings. Suddenly, there is a severe driver shortage in major cities, causing surge pricing to skyrocket, which makes average passengers abandon the app for competitors.

🧠 **The Inquiry Challenge for Students:**

Do not tell them the climax. Give them the scenario and say: *"Draw the feedback loops. If this software operates inside a human ecosystem, what hidden connections exist between a passenger's ego, a driver's livelihood, and the company's surge-pricing algorithm?"*

- **Goal:** Students must move from *predicting outputs* to *mapping emergence*. They must use **discernment** to realize that code interacts with human psychology, creating non-linear consequences.

Group Exercise 2: The Haunted AI Legacy System (First-Principles Thinking)

The Scenario:

A major airline's flight-scheduling software is failing during summer storms. The system was built entirely by an advanced AI assistant three years ago. No human engineers fully understand how it works; they have only ever patched it by asking the AI to *"fix the bug"* whenever it crashed. The current code is a bloated, incomprehensible mess of overlapping rule dependencies. Tomorrow, a massive hurricane is hitting the East Coast. [1]

- **The Reductionist Trait (The Trap):** Students will naturally try to act like a traditional automated debugger. They will suggest looking at the specific log files of the last crash, guessing which variable failed, and prompting an LLM to patch that specific module.
- **The Systems Reality (The Climax):** The AI-generated code relies on a fundamentally flawed premise: it assumes that planes always return to their scheduled base at night. In a hurricane, planes are diverted randomly, breaking the AI's core mathematical logic. No amount of local patching will ever fix it.

🧠 **The Inquiry Challenge for Students:**

Forbid them from proposing patches. Instruct them: *"Strip away the software entirely. What are the absolute, unalterable truths (First Principles) of airline physics? (e.g., A pilot can only fly \(X\) hours; a plane cannot be in two places at once; passengers require \(Y\) connection time). Rebuild the scheduling logic on a whiteboard using nothing but these fundamental truths, ignoring how the current software does it."*

- **Goal:** Students learn **metacognition and discernment**. They realize that when dealing with complex or AI-generated structures, you cannot accept the current state as "truth." You must break it down to its atomic realities to build a resilient conceptual model.

Group Exercise 3: The Smart City Power Grid Game (Agility & Emotional Intelligence)

The Scenario:

The class is divided into 4 competing teams representing different municipal sectors of a futuristic smart city: **The Hospital Network**, **The Industrial District**, **The Residential Smart-Homes**, and **The Autonomous Transit Grid**. All 4 sectors pull from a shared, AI-managed power grid.

- **The Setup:** A severe heatwave hits the city. The AI power grid detects a critical energy shortage and initiates an automated bidding war for electricity.
- **The Reductionist Trait (The Trap):** Each student team behaves like an isolated agent. They design a deterministic strategy to optimize *only* their own sector (e.g., the Industrial District bids all its capital to keep factories running; the Residential sector overrides local thermostats to keep citizens cool).
- **The Systems Reality (The Climax):** Because every team optimizes locally, they cause a systemic blackout. The transit grid dies, leaving factory workers stranded. The hospital's backup generators fail because the surrounding grid is entirely unstable. Everyone loses.

🧠 **The Inquiry Challenge for Students:**

Stop the exercise halfway through the chaos. Force the teams to negotiate a shared protocol without changing their internal sector goals. *"How do you design an adaptive, collaborative software agreement (an API contract) between your sectors that balances individual greed with systemic survival?"*

- **Goal:** This explicitly tests **Agility and Emotional Intelligence**. Students experience the friction of competing dependencies. They learn that system architecture is heavily bound by human negotiation, trust, and shared communication protocols.

IV. Field Notes - From Personal Experience

Connect each abstract concept to real TPM experience

After reductionism vs. systems thinking:
Tell a 5-minute story about a project where the technical solution was correct, but the rollout failed because of dependency, ownership, stakeholder, migration, or adoption issues.

After API contracts/human chaos:
Explain that in enterprise projects, an interface contract is not just a technical agreement. It also encodes ownership, escalation path, SLA, monitoring responsibility, rollback authority, and political trust.

After AI legacy system:
Discuss how teams inherit systems nobody fully understands, and the real skill is often not “fix the bug,” but “reconstruct the conceptual model.”

V. Grading Rubric